

HW-2 Write-up

Problem 1B

As referenced in Section 3 and Figure 2 of the paper, each key of the input representations correspond to:

1. **input_ids**: These represent tokenized input sequences using WordPiece embeddings with a 30,000-token vocabulary. Each token (word/subword/punctuation mark) is mapped to a unique ID from this vocabulary. The special tokens **[CLS]** and **[SEP]** corresponding to **101** and **102** respectively mark sequence boundaries (start and end).
2. **token_type_ids**: These are the *segment embeddings* referenced in the paper that differentiate between two sentences in a pair. If multiple sentences are provided, the first sentence gets **0**s and the second gets **1**s
3. **attention_mask**: This mask tells BERT which tokens should be attended to (**1**) and which should be ignored (**0**), based on whether they are padding tokens or not. This ensures BERT avoids unnecessary computation on padded positions.

Problem 1C

The hyperparameter tuning procedure for the BERT tiny model as documented in the official BERT GitHub repository is described below:

1. **Model Variants**: The release includes 24 smaller "student" BERT models with varying numbers of layers (L) and hidden sizes (H).
2. **Training Objective**: The smaller BERT models are trained using the same procedure for standard BERT training, including the model architecture and training objective.
3. **Fine-tuning Procedure**: The smaller BERT models are fine-tuned in the same manner as the original BERT models via knowledge distillation. Labels were generated by larger and more accurate 'teacher models'.
4. **Hyperparameter Training for Glue Benchmark**: For each task in the GLUE benchmark, the best fine-tuning hyperparameters were selected from the following lists:
 - **Batch Sizes**: 8, 16, 32, 64, 128
 - **Learning Rates**: 3e-4, 1e-4, 5e-5, 3e-5

Each hyperparameter was trained for 4 epochs

5. **Evaluation**: The performance of the models on the GLUE benchmark was evaluated using for the selected hyperparameters. The results for different model variants (e.g, BERT-Tiny, BERT-Mini, BERT-small, BERT-medium) were tabulated in terms of accuracy and the best was noted.

Problem 3A

	Validation Accuracy	Learning Rate	Batch Size
Without BitFit	0.887	0.0003	128
With BitFit	0.635	0.0003	16

Problem 3B

	# Trainable Parameters	Test Accuracy
Without BitFit	4,386,178	0.876
With BitFit	3,074	0.636

In my experiment, we see that with BitFit, the number of trainable parameters decreases from over 4 million to around 3,000. However results suffer significantly when fine-tuning the BERT Tiny model. Whereas training with all parameters yielded 87.6% accuracy, using BitFit reduced accuracy to 63.6%.

In the paper written by Zaken et al. (2020), we see that using BitFit did not reduce accuracies significantly. Particularly in Table 2, we see that accuracies actually marginally increased from Full-FT to BitFit for BERT Base and BERT Large, while slightly decreasing in the case of RoBERTa Base.

Thus, BitFit may not be suited for small models, such as BERT Tiny which was fine-tuned here, in contrast to the larger models fine-tuned in the paper. It is possible that the dataset used also has an effect on the difference in performance between BitFit and Full-FT. The primary advantage for BitFit is that it reduces the number of parameters significantly reducing the computational intensity of training. Further studies could evaluate the difference between larger models with BitFit vs smaller models Full-FT, where the number of trainable parameters and computational intensity is roughly the same.